STANSE: Bug-finding Framework for C Programs

Jan Obdržálek, Jiří Slabý and Marek Trtík

Faculty of Informatics, Masaryk University, Brno, Czech Republic
obdrzalek@fi.muni.cz, slaby@fi.muni.cz, trtikm@mail.muni.cz

Abstract. Stanse is a free (available under the GPLv2 license) modular framework for finding bugs in C programs using static analysis. Its two main design goals are applicability on large software projects like Linux kernel and extendability with new bug-finding techniques with a minimal effort. There are currently three bug-finding algorithms implemented within Stanse: AutomatonChecker checks properties described in an automata-based formalism, ThreadChecker detects deadlocks among multiple threads, and ReachabilityChecker looks for unreachable code. Stanse has been tested on the Linux kernel, where it has found dozens of previously undiscovered bugs.

1 Introduction

Bug-finding based on static analysis have finally come of age in the last decade. One of the papers to really stir interest was [2], showing that static analysis can efficiently find many interesting bugs in a real-world code, as demonstrated on the Linux kernel. This work eventually led to a successful commercial tool called Coverity [7]. Over the years, several other successful tools, like CodeSonar [6] or Klocwork [9], appeared. However, such fully-featured tools are neither free to obtain, nor is their code available (e.g. for developing new algorithms or tailoring the existing tools to specific tasks). The existing free tools are usually severely limited in what they can do (e.g. Uno [12], Sparse [11], Smatch [10]). One notable exception is FindBugs [4, 8], a successful tool working on Java code. Stanse is intended to fill this gap for C language. It can be seen in two ways:

1. Stanse is a robust framework (written predominantly in Java) for implementing diverse static analysis algorithms. An implemented algorithm can be immediately evaluated on large real-world software projects written in C as the framework is capable to process such projects (for example, it can process the whole Linux kernel). An implementation of such an algorithm within the framework is called *checker*.
2. As Stanse contains three checkers already, it can be also seen as a working static analysis tool.

The rest of the paper is structured as follows. Section 2 describes the functionality provided by the framework, while Section 3 is devoted to the three

existing checkers. Section 4 presents some results of Stanse on the Linux kernel. The last section summarizes the basic strengths of Stanse and mentions some directions of its future development.

2 Framework Functionality

The functionality provided by the Stanse framework can be roughly separated into several stages. First, the framework reads in the configuration including source file list and checkers to be used. The source files are then parsed into a set of *internal structures*, namely *abstract syntax trees (ASTs)*, *control-flow graphs (CFGs)*, and *call-graphs*. Next, checkers are run on these structures and error reports are collected and presented.

2.1 Selecting and Parsing Source Files

There are several ways to tell Stanse what source files to analyze. Besides the standard possibilities (a single given file, all files from a given directory, all files listed in a given file), Stanse can also derive all the necessary information from a project Makefile. In this case, Stanse also remebers compiler flags for preprocessing purposes. This functionality is inspired by the Sparse [11] tool.

Stanse can process source files written in C, more specifically the ISO/ANSI C99 standard and most of the GNU C extensions. Before parsing, the sources are preprocessed by the standard GNU C preprocessor. The code is then parsed into the mentioned internal structures: a call-graph, a CFG for each function, and an AST for each code piece corresponding to a node of some CFG. All these structures can be dumped in a textual or graphical form.

As mentioned earlier, the design of the framework is geared towards analyzing large software projects. Such projects consist of hundreds or thousands of modules (compilation units). In practice, it is often impossible to store all the corresponding internal strucures in the memory. The Stanse framework therefore applies automatic *streaming* of the internal structures, currently on a module basis. Instead of parsing all source modules at the beginning, a module is streamed in only when Stanse needs to access some internal structure corresponding to the module. In other words, the internal structures are constructed in a lazy manner – we call them *lazy internals*. If the memory occupied by internal structures exceeds a given limit, some internal structures have to be freed before another module is streamed in. The structures to be freed are selected by an LRU (least-recently-used) approach: Stanse discards all internal structures of the module whose structures are not accessed for the longest time. If the discarded structure is later accessed again, the corresponding module is streamed back in. Both laziness of internal structures and streaming are completely hidden to checkers.

Probral jsem to se zenou a oba si myslime, ze "transparent" by znamenalo, ze checkery mohou ten proces sledovat, coz neni pravda.

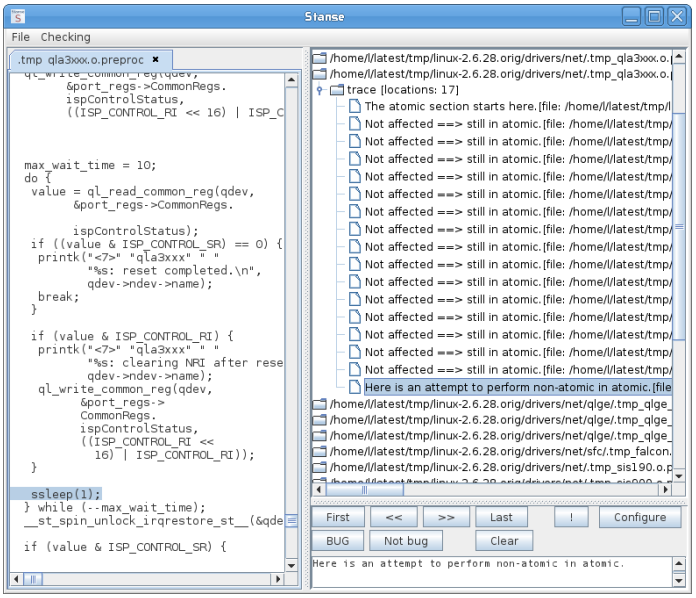

Fig. 1. The error trace GUI browser.

2.2 Traversing the Parsed Code

Althought a checker can process the internal structures in an arbitrary way, most of them walk through CFGs using some standard strategy. To prevent unnecessary reimplementations, the most important and heavily used traversal methods are implemented directly inside the framework. With this functionality, one can implement a new checker (or its part) by specifying

- whether it should go through CFGs forwards or backwards, breath-first or depth-first,
- if the interprocedural walk-through should be performed or not (if not, the function calls are ignored), and
- a method (callback) to be called for each visited node in a CFG.

This makes implementation of new algorithms extremely simple.

2.3 Processing Errors

Once a checker finds an error, it reports back a commented *error trace* (a path in the analyzed code) demonstrating this error. Stanse provides several possibilities to present the error traces: they can be printed to the console, displayed using a built-in error trace GUI browser (see Figure 1), or saved to an external file in XML format. This XML file has a wide variety of possible applications. For example, we supply a tool transforming the XML file into an SQLite database complemented with a web interface allowing to browse errors in the database

via a web browser. Using the web browser or the built-in error GUI browser, one can mark errors as real bugs or or false-positives. Stanse also provides various statistics of errors like number of errors per file and checker, frequency of errors of the same kind, percentage of false-positives (based on user feedback), etc.

2.4 Other Features

The framework provides some other features. For example, it supports configuration files describing both the files to be analyzed and the checkers to be used. Further, the framework supports parallel execution of more checkers (in threads). Note that the internal structures are shared by all running checkers. Moreover, Stanse can run in a command line mode or via a fully featured GUI. If properly configured, Stanse can be used as a "push-the-button" tool. Finally, the Stanse framework has been designed as a Java library, so it can be easily transformed into a static analysis plug-in for Eclipse or NetBeans IDE.

3 Checkers

In this section we describe the three currently available checkers.

AutomatonChecker is heavily influenced by [2]. It takes, as an input, a set of finite-state automata that describe some properties which we want to check. Properties like locking discipline, interrupt management, null pointer dereference, dangling pointers and many others can be handled this way.

Compared to the implementation described in [2] and [3], our technique differs in several aspects. In particular, we do not use metacompilation, automata are not input-language specific (a pattern matching is used instead), and the interprocedural analysis is done in the context of a single input file.

ThreadChecker aims to check for possible deadlocks in concurrent programs. The technique is based on the notions of locksets of [5] and deadlock detection by looking for cycles in *resource allocation graphs (RAGs)*. ThreadChecker first tries to identify the parts of the code which can run in parallel, as different threads. Then it builds a set of *dependency graphs* for each such a thread. A dependency graph statically represents the possible locksets during one execution of a thread. Dependency graphs are then combined and transformed to RAGs. If some RAG contains a cycle, an error is reported.

ReachabilityChecker searches CFGs for unreachable nodes. These are then reported as warnings or errors, depending on importance (e.g. superfluous semicolons are less important than unused branch). The primary goal of ReachabilityChecker is to demonstrate the simplicity of a new checker implementation using the framework features described in Subsection 2.2: the code of the checker has less than 200 lines including the mentioned error/warning classification and many strings and comments.

```
static void untag_chunk(struct node *p) {
 spin_lock(&entry->lock);
 ...
 new = alloc_chunk(size);
 ...
 spin_unlock(&entry->lock);
}
static struct audit_chunk *alloc_chunk(int count) {
 ...
 chunk = kzalloc(size, GFP_KERNEL);
```

Fig. 2. Bug found in the audit code

4 Results – Linux Kernel

We are exercising the Linux kernel by Stanse since 2.6.28 was released back in 2008. It was the first release we decided to test the usability of Stanse on. One reason to choose the Linux kernel is that it represents a large and freely available codebase, where the absence of bugs is of great concern. Moreover, the Linux kernel fully exercises most features of ANSI C99 and GNU C extensions. The other motivation is to provide a free tool for Linux kernel developers.

Currently, Coverity Prevent [7] is daily run on the current development branch of the Linux kernel, and the results are reported back to developers. However, developers kept asking for a tool with which their code will be checked before it appears upstream. Moreover, the kernel code is very specific and the tool should consider these specifics.

We used all three checkers described above. These checkers are not kernel specific and only the configuration for AutomatonChecker has to be tailor-made to look for the type of bugs commonly found in the kernel code. The properties checked by the AutomatonChecker in the kernel are:

- correct pairing of functions like balanced locking, check for correct reference counting,
- memory/pointer related bugs like dereferencing null pointers and
- atomic section restrictions to find if someone can cause deadlocks by sleeping inside spinlocks or interrupt handlers.

With this configuration, the running time of Stanse on a common desktop machine with 2 cores and 4 GiB of memory is under 100 minutes. With streaming, the memory usage of the Java process oscillates between 400 and 1000 MiB in case of forced garbage collector executions. The standard Stanse Makefile support was used and we did not have to manually tweak the source code in any way in order to be processed by Stanse.

In the 2.6.28 release, more than 70 bugs[1] were reported to kernel developers, most with appropriate patches, and fixed in the next kernel release. Some of these bugs remained undiscovered for seven years[2]. This is despite the kernel

[1] A list of confirmed bugs is available at `http://stanse.fi.muni.cz/bugs.html`
[2] `http://lkml.org/lkml/2009/3/11/380`

Automaton	Errors	Assorted	Bugs	FP	Ratio
Pairing	266	58	65	143	31.3 %
Memory	86	1	48	37	56.5 %
Atomic	35	1	16	18	47.1 %
Overall	387	60	129	198	39.4 %

Table 1. Errors found by AutomatonChecker in the 2.6.28 Linux kernel. Assorted = Neroztrizene = nepodarilo se rozhodnout.

being daily checked[3] with Coverity Prevent and we believe it is due to the filters they use.

Now we present one bug which we found in the code reporting security incidents. An excerpt from the particular code is in the Figure 2. `untag_chunk` functions creates a critical section by a spinlock and calls `alloc_chunk` which in turn can sleep in `kzalloc`. This can, under certain circumstances, lead to deadlock of the whole system. The bug was confirmed and fixed.

The false positive (FP) rate oscillates among the different bug types, the overall ratio is that 40 % of all reported errors are real bugs. This result is quite low, however similarly to [1], it is without any thorough false-positive filtering techniques (cf. Future Work section). The more detailed results about the AutomatonChecker are in the Table 1, one row per each configuration.

The current kernel versions are checked with other tools like Coccinelle or Smatch. Mostly these tools find simple pattern matching style properties like lock-unlock and are successful at that. We continue to check the kernel releases with Stanse too and, thanks to interprocedural analysis, we still are able to find errors on daily basis.

The most convenient advantage on these tools is their availability. The developers or testers can take them whenever they want and check their code instantly, because they are freely available and they can handle the kernel code. Further these tools need near no configuration (either because they provide predefined one or there is nothing to configure) and are ready to use out of the box.

5 Conclusions and Future Work

Stanse is a free Java-based framework for simple implementation of bug-finding algorithms based on static analysis. The framework can process large-scale software projects written in ISO/ANSI C99. Stanse does not currently use any new techniques – its novelty comes from the fact that (to our best knowledge) there is no other non-commercial open-source framework with comparable properties. Stanse is available on `http://stanse.fi.muni.cz`.

Even if Stanse with its three simple bug-finding algorithms has already found more than 130 real bugs in Linux kernel and this number is growing every week, there are still many things that should be improved. For example, we

Jirko, je to pravda?

[3] `http://marc.info/?l=linux-kernel&m=125856033627134&w=2`

are currently working on C++ support (note that the framework can handle any programming language for which a frontend is written). Further, we plan to provide Stanse in the form of a plug-in for Eclipse or NetBeans IDE. A lot of work can be done in the area of automatic false-alarm filtering and error importance classification. Independently of new features development, we would like to speed up the framework as well. More precisely, we intend to replace the current parser written in Java by an optimized parser written in C, to replace the XML format of internal structures by something more concise, to add a support for function summaries, etc.

Acknowledgements Jan Kučera is the author of the ThreadChecker. We would like to thank Linux kernel developers, and Cyrill Gorcunov for Stanse alpha testing and useful suggestions. All authors are supported by the research centre Institute for Theoretical Computer Science (ITI), project No. 1M0545.

References

1. T. Ball, B. Hackett, S. Lahiri, S. Qadeer, and J. Vanegue. Towards scalable modular checking of user-defined properties. *Verified Software: Theories, Tools, Experiments*, pages 1–24, 2010.
2. D. Engler, B. Chelf, A. Chou, and S. Hallem. Checking system rules using system-specific, programmer-written compiler extensions. In *OSDI'00*, pages 1–16, 2000.
3. S. Hallem, B. Chelf, Y. Xie, and D. Engler. A system and language for building system-specific, static analyses. In *PLDI'02*, pages 69–82. ACM, 2002.
4. D. Hovemeyer and W. Pugh. Finding bugs is easy. In *OOPSLA'04*, pages 132–136. ACM, 2004.
5. J.W. Voung, R. Jhala, and S. Lerner. RELAY: static race detection on millions of lines of code. In *ESEC-FSE'07*, pages 205–214. ACM, 2007.
6. CodeSonar. `http://www.grammatech.com/products/codesonar/`.
7. Coverity. `http://www.coverity.com/products/`.
8. FindBugs. `http://findbugs.sourceforge.net/`.
9. Klocwork. `http://www.klocwork.com/products/`.
10. Smatch. `http://smatch.sourceforge.net/`.
11. Sparse. `http://www.kernel.org/pub/software/devel/sparse/`.
12. Uno. `http://spinroot.com/uno/`.

6 What's Missing

There are several points we miss which were commented in the past reviews. They include:

- soundness, completeness
- scalability, applicability
- full bugs description

Markovo summary:

- e jde v prvn ad o framework pro statickou analzu. Tedy e nabzme funkn balk kdu, do kterho se d s minimlnm silm naimplementovat jist konkrtn nov technika.
- Co tedy framework nabz: Configuration, LazyInternals(ASTs,CFGs,Streaming), Inter/Intra-analzy, Traversovn, Statistics, GUI.
- Pro otestovn frameworku mme napsno nkolik checker (Automaton,...). Stanse jsme byli schopni pustit na cel kernel Linuxu a nalzt v nm nkolik chyb.

Ty interni struktury odpovidaji proceduram nebo modulum? A plati modul=file? Jak je psano niz, modul tady odpovida compilation unit, coz je jeden soubor. CFG odpovida jedne funkci, AST je rozkouskovane po uzlech CFG a call-graph se (momentalne – kdybysme neorezavali Configuration jen na jeden soubor, tvoril by se call-graph pro vsechny v Configuration) tvori v ramci jednoho souboru. Kdyz se parsuje soubor, tak pro vsechny funkce se vyrobi CFG s opdovidajicimi AST uzly a take se vyrobi call-graph vsech funkci.

V cem spociva podpora interproceduralni analyzy? V tom, ze si muzu rict, ze do volanych struktur bud vlezu, nebo budu volani ignorovat? JS: rekl bych, ze ano.

Nemel bych nekde rict, kterou strukturu chci primarne prochazet (call-graf, AST, CFG)? JS: prochazet se da jen CFG. U AST to asi nema smysl, u call-graphu asi ano, ale na to neni pruchodce (nikdo ho zatim nepotreboval).

Kdyz nahraju modul/soubor, kde ta analyza zacne? Projdu kazdou proceduru? JS: ano, AC prochazi kazdou, TC si vybira ty, ktere nejsou volany nikym jinym v ramci souboru (tzv. pocatecni funkce)

Predpokladam, ze interni struktury jsou provazany: treba uzly v CFG maji vazbu na odpovidajici podstromy v AST. Je to tak? JS: uplne presne

Prilepime sem neco jako: ***Note that the checkers are independent of each other, they can be executed sequentially or in parallel.*** **A kdyz mam vic checkeru, to se pro kazdej checker zvlast vytvari internal structures? A novej checker se musi do toho frameworku nejak registrovat?** JS: Note dava smysl. Internals jsou spolecne pro vsechny a jsou pro checkery read-only. Registrace checkeru spociva v pridani jednoho radku do CheckerFactory – ano, musi se “registrovat”